Assignment Description for Project 6 (Virtual Machine I)

Objective

Get a deep understanding of the operation of the VM and of the process of compiling VM code into assembly code, which essentially corresponds to what is done in the 'backend' of a modern compiler.

Contract

Create a VM language parser and a VM to Hack assembly compiler by extending the Python source code provided. Implement 1 member function of the VMParser class and 2 member functions of the CodeWriter class.

The functions in Python are the following:

- Parser._Parse(self)
- CodeWriter.WritePushPop(self, commandType, segment, index)
- CodeWriter.WriteArithmetic(self, command)

We recommend that you first complete the implementation of the *_Parse()* function in *hvmParser.py* so that it can handle all types of commands. You can test the implementation by creating a VM code file that contains one of each command type, or use the given *BasicTest.vm*, and then by looking at the result of the parsing.

Once the parser is correct you can start with the first code writing function (*WritePushPop()*) and then you can continue with the second code writer function (*WriteArithemtic()*). We recommend that once you are done with the first function you test your compiler on the MemoryAccess program, and once you are done with the second function, you test your compiler on the StackArithmetic program provided for Chapter 7 using the CPU emulator (5 tests).

Resources

You will extend the provided VMParser and the CodeWriter classes. You find the classes in the corresponding .py files with the same name. You are also provided the hvm.py file, which contains the main() function of the code to be executed.

The provided code consists of 4 files:

hvm.py: This file contains the 'main' function that uses the functionality implemented in the classes VMParser and CodeWriter to compile a set of input VM files into a single ASM file. The program function takes 1 optional command line argument and one mandatory command line argument. The mandatory argument is either the name of the VM file to be processed or the name of a directory, in which case all all .VM files in the directory will be processed. The syntax for calling the script is

```
python hvm.py [-nosysinit] <sourcefile(s)>
```

For Project 6 you will have to invoke the compiler with

```
python hvm.py -nosysinit source.VM
```

which will compile the VM file source into the ASM file source.ASM, and it would not insert a call to 'sys.Init'. You will deal with the implementation of calling 'sys.Init' in Project 7.

hvmParser.py: Implementation of the parser.

hvmCommands.py: Definitions of the command type values.

hvmCodeWriter.py: Implementation of the CodeWriter class. The implementation of 9 functions is missing. Each of the nine functions is responsible for compiling a particular VM command type (there are 9 types). Out of these 9 functions you will have to implement 2 in this project. We recommend that you implement the 2 functions in the following order:

1. CodeWriter::WritePushPop
2. CodeWriter::WriteArithmetic

The provided code is consistent with the interface specification provided in Chapters 7 in the book. It contains the implementation of the functionality that is not strictly related to the task of compiling VM language code into assembly language code.

For the rest of the resources (VMEmulator and Hardware emulator) please refer to Chapter 7.5 of the book.

Hints

- Remember that the task is to "translate" from VM instructions to Hack Assembly with the use of Python.
- Understand the given functions in *hvmCodeWriter.py*, such as *_WriteCode(self, code)* and *_UniqueLabel(self)*, and use them in your implementation.
- Create helper functions for repeated sequences, such as push and pop operations on the stack, to achieve higher readability and reduction of possible fault points.
- Use print frequently to visually ensure the code is executed as expected and that variables have the correct values, e.g. *print('command is now: ', command)*

Submission

- You need to submit all the python files needed to run the translator, that is *hvm.py*, *hvmCodeWriter.py, hvmCommands.py* and *hvmParser.py*.
- In one zip-archive, include the source files and the grade sheet in Excel. Please use the convention *Project06-familyname-firstname.zip* for the filename.

Only students' own work can be submitted. Copying of solutions and any other forms of cheating will be reported for disciplinary action!